

RESEARCH

Open Access



Self-supervised graph neural networks for network intrusion detection in cloud-edge collaboration environments

Haiyang Diao¹, Xiang Li¹, Wan Zhang¹, Yan Sibyl Yu³, Ye Wang¹ and Jing Zhang^{1,2*}

Abstract

Cloud-edge collaboration enables intelligent services by combining powerful cloud computation with latency-sensitive edge analytics, but it also widens the attack surface as distributed nodes exchange model updates and traffic over heterogeneous networks, heightening privacy and security risks. Graph neural networks (GNNs) are promising for network intrusion detection (NID) by capturing topological and feature dependencies, yet most approaches rely on supervised, centralized training, limiting scalability and privacy in distributed settings. Additionally, these methods aggregate node and edge features uniformly, overlooking differences among malicious traffic types and limiting detection. To address these challenges, this paper proposes Self-Supervised Graph Multi-Head Attention Networks (SSGMHAN) for NID. The proposed method employs structure-aware graph contrastive learning to form positive and negative pairs and learn from large-scale unlabeled traffic, yielding representations robust to structural perturbations. A multi-head node-edge attention module dynamically emphasizes attack-specific nodes and edges, producing more expressive embeddings. On three benchmarks, SSGMHAN surpasses state-of-the-art unsupervised NID methods, offering an effective self-supervised framework with attention-based aggregation.

Keywords Network intrusion detection, Graph neural networks, Self-supervised learning, Multi-head attention mechanism

Introduction

The convergence of cloud computing, edge computing, and artificial intelligence has fostered cloud-edge collaborative systems, where large-scale cloud models work in tandem with lightweight edge inference modules. While this architecture promises low-latency analytics and efficient workload distribution, its deployment faces challenges such as cross-domain latency, demand fluctuations, resource uncertainty [1], and high data transmission costs [2]. Furthermore, this collaboration expands the attack surface, exposing infrastructure and users to diverse network risks [3] and escalating privacy and security concerns [4–7]. In this context, defenses are severely challenged by the rapid evolution of network

*Correspondence:

Jing Zhang
jingz@seu.edu.cn

¹School of Cyber Science and Engineering, Southeast University, No. 2 SEU Road, Nanjing 211189, China

²Ministry of Education, Engineering Research Center of Blockchain Application, Supervision and Management (Southeast University), No. 2 SEU Road, Nanjing 211189, China

³Business School, Macquarie University, Wallumattagal Campus, Macquarie Park, NSW 2109, Australia

attack patterns, the instability of traffic features, and the strong hybrid nature of intrusions that blend with legitimate flows. To address these challenges, numerous Network Intrusion Detection (NID) methods have been developed.

Existing methods principally fall into signature-based and anomaly-based categories [8]. Signature-based methods [9] match flows with predefined rules, which are rigid and fail against novel threats. Meanwhile, anomaly-based methods relying on traditional machine learning (ML) [10] effectively use feature engineering but struggle to adapt to new, complex attack types due to the static nature of handcrafted features. Deep learning-based Network Intrusion Detection Systems (NIDSs) [11] have improved detection by automatically extracting high-level features. However, they predominantly operate under a supervised paradigm. This approach necessitates massive amounts of accurately labeled data [12, 13], creating a significant bottleneck. With the exponential growth of Internet of Things (IoTs) devices, annotating massive network traffic is prohibitively time-consuming and costly [14], sometimes infeasible [15], and existing supervised methods remain brittle when facing unknown attacks.

Recently, graph neural networks (GNNs) [16] represent a promising technology in NIDSs [17]. As we know, extracting the most pertinent features from raw network flows is crucial for intrusion detection. Many traditional machine learning-based NID methods explore network flow features from various perspectives, but they frequently neglect to consider the topological relationships between network flow information [18]. And graph structures are widely used to represent the topological relationships of network traffic [19, 20]. In essence, network flows can be naturally represented as a graph, where internet protocol (IP) addresses and port numbers are nodes, and network flows traversing between different IP addresses and ports serve as edges connecting the nodes [21]. Furthermore, treating both nodes and edges as embedding targets and modeling their interactions jointly has been shown to capture complementary structural and semantic cues in heterogeneous graphs [22].

Through collecting information about neighboring nodes in the vicinity of a target node, GNN-based NID methods obtain more representative high-dimensional embeddings for nodes through diverse aggregation techniques [23], which subsequently are applied to the downstream classification tasks. In this procedure, sampling and aggregation are of particular significance. For example, common GNN methods applied to NID, such as GraphSAGE [24], entail the mean-value aggregation of node features and edge features within the neighborhood in the aggregation stage, without distinguishing between normal and different categories of malicious

traffic. However, high-degree hubs in scale-free networks typically act as “super-spreaders” [25]. Xiao et al. [26] also demonstrate that indiscriminate aggregation propagates messages along adversarial edges and biases decision boundaries. Therefore, one should prioritize nodes likely to launch malicious attacks and edges indicative of malicious traffic, aggregating these features to learn more discriminative embeddings for malicious-traffic identification [27].

To address the heavy reliance on annotated data, the underutilization of edge-centric structural information, and the limitations of average feature aggregation in existing methods, we proposed a novel Self-Supervised Graph Multi-Head Attention Network (SSGMHAN) for NID in previous work [28]. It adaptively assigns weights to different nodes and edges in the node feature aggregation phase by introducing a multi-head attention mechanism into GNNs, thereby enhancing its ability to identify different types of malicious traffic. In contrast, this study further introduces a dynamic edge pruning strategy based on these attention weights to address the lightweight requirements of edge devices in cloud-edge collaborative systems. This strategy effectively eliminates redundant structural information, thereby reducing the computational overhead for resource-constrained edge deployment. The proposed model employs a graph contrastive learning method that enables self-supervised learning, which can reveal the underlying structure of the data without the necessity for human annotation or a priori knowledge. Furthermore, by introducing feature-level random nodes and edge perturbations into the contrastive learning framework, the robustness of this method to real-world noise is improved. Compared with intrusion detection methods based on supervised learning, the proposed method expands the detection coverage of intrusion detection for previously unseen malicious activities in the network while reducing labor and time costs. The contributions of this paper are as follows:

- We propose a novel self-supervised GNN-based NID method, namely SSGMHAN, to identify malicious traffic and its specific attack types. The proposed method achieves self-supervised graph representation learning through generative subgraph contrast (GSC), which is based on the adaptive generation of positive and negative samples utilizing random feature-level node and edge perturbations and combines edge and topological features.
- In the encoder component of the SSGMHAN model, we incorporate the multi-head attention mechanism into the graph neural networks, thereby facilitating the adaptive assignment of weights to disparate edge and node features. Leveraging these learned attention distributions, we further propose a

dynamic attention edge pruning strategy to generate a lightweight model more suitable for resource-constrained cloud-edge collaborative systems.

- We conducted extensive experiments on three benchmark network intrusion detection datasets and demonstrated that our proposed model outperforms most current self-supervised learning models on multi-class classification tasks.

The structure of this paper is as follows: Section 2 offers a concise review of related work. Section 3 introduces the fundamental concepts relevant to this study. Section 4 elaborates on the technical aspects of the proposed approach. Section 5 presents and analyzes the experimental findings. Finally, Sect. 6 summarizes the conclusions of this research.

Related work

Self-supervised NID methods

Traditional supervised learning-based NID methods [29, 30] are contingent upon accurately labeled training data, which can hardly meet the requirements of detecting fast evolving unknown attacks. To address this issue, Song et al. [31] proposed a self-supervised learning-based NID method. Their method involves first generating pseudo-normal traffic with noise using a generator, and then training a detector to differentiate these anomalous traffic patterns from normal traffic. Wang et al. [32] incorporated a mask context reconstruction module into the transformer-based self-supervised learning scheme, proposing a RUIDS method. Their objective is to learn intrinsic relationships within the temporal context. With the mask context reconstruction module, the model learns a more robust representation that is less susceptible to anomaly contamination and less sensitive. Additionally, on the unsupervised side, reconstruction-based methods learn normal traffic patterns and flag deviations by modeling reconstruction error over time. CAE-IF [33] captures temporal regularities in this error signal and uses data-driven anomaly scoring instead of fixed thresholds, demonstrating strong performance across diverse datasets.

GNN-based NID methods

Many early deep learning-based NID methods overlook the topological structures of network flows and internet protocols (IPs). GNNs offer a promising solution by directly representing system architecture as nodes and edges of a graph, enabling more effective identification and categorization of attack patterns [34]. Zhou et al. [35] proposed using graph convolutional networks (GCNs) to detect peer-to-peer botnet nodes. They generated botnet traffic by blending real network traffic with botnet connections and applied GCNs for classification. Xiao et al.

[36] introduced an anomaly detection model leveraging graph embedding techniques based on network flow representations. Their model transforms network flow data into first-order and second-order graphs, utilizing the generated graph embeddings and feature values to train a random forest classifier. Similarly, Lo et al. [37] proposed an E-GraphSAGE model based on GCNs for Internet of Things (IoTs) intrusion detection. The model constructs the network traffic topology through source and destination nodes and applies convolutional operations to the traffic features as edge attributes to obtain updated embeddings for malicious traffic identification. To address the issue of small and imbalanced attack datasets, Zhou et al. [38] proposed a hierarchical adversarial attack generation method, which utilizes shaded GNNs and salient graph techniques to generate adversarial examples with minimal perturbation by effectively identifying and modifying key feature elements. SCoAMPS [39] is a semi-supervised graph contrastive learning method that integrates multi-encoder views, pseudo-label-based positive and negative selection, and an associative memory network (AMNet) to alleviate label scarcity.

Self-supervised GNN-based NID methods

Self-supervised GNN-based methods can have the advantages of both self-supervision and GNNs. Caville et al. [40] proposed the Anomal-E model, which utilizes E-GraphSAGE in unsupervised representation learning for network intrusion detection by maximizing mutual information between edge representations and employing various methods like Principal Component Analysis (PCA) and Isolation Forest (IF) for anomaly detection. Nguyen et al. [41] introduced a novel traffic-aware self-supervised learning method for intrusion detection in IoT networks, namely TS-IDS. The authors enhanced the graph representation through attribute-based assisted self-supervised learning (SSL) and conducted experiments on unlabeled datasets to demonstrate the potential of their proposed framework. Xie et al. [42] introduced CoGN, a network intrusion detection approach incorporating a collaborative contrast mechanism. This method integrates embedded learning pre-training with unsupervised detection algorithms. By leveraging a graph neural network, the model captures both local and higher-order features from network patterns and meta-path perspectives. These two perspectives interact and reinforce each other, achieving high-level traffic embeddings. Xu et al. [43] introduced a NID method employing generative subgraph contrastive learning, namely NetFlow-Edge Generative Subgraph Contrast (NEGSC). It constructs subgraphs from sampled nodes, aggregates edge features using an attention mechanism, and considers subgraph pairs sharing the same central node as positive samples, while those with different central nodes are treated as

negative samples. The NEGSC model is optimized by computing the contrastive loss within these pairs. Compared with their methods, this study first proposes to integrate the multi-head attention mechanism of the graph into the self-supervision framework, so as to make the generative samples have better embedding representation, and ultimately significantly improve the model performance.

Recent self-supervised graph neural network (GNN) approaches have demonstrated strong potential in network intrusion detection (NID). To compare state-of-the-art (SOTA) methods with the improvements introduced by SSGMHAN, Table 1 summarizes the core ideas, main advantages, and limitations of four representative self-supervised GNN frameworks.

Prerequisite knowledge

Graph attention networks

Velickovic et al. [48] proposed the Graph Attention Networks (GATs), a method that adaptively assigns weights to the edges between nodes to capture the relationships between them, thereby capturing more important information during the node aggregation phase. GAT is a highly significant approach in the current field of Graph Neural Networks [49], as it calculates the relevance between nodes through a self-attention mechanism, which determines the influence of each neighbor during information propagation. When aggregating information, a node considers both the features of its neighbors and their importance, allowing it to more flexibly capture key relationships within the graph structure. The feature update process for node i in the GAT can be represented as follows:

$$h'_i = \text{LeakyReLU} \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} W h_j \right), \quad (1)$$

Here, the attention weight α_{ij} can be obtained as follows:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [W h_i \| W h_j]))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LeakyReLU}(\mathbf{a}^T [W h_i \| W h_k]))} \quad (2)$$

where W is a learnable weight matrix, $\mathcal{N}(i)$ is the neighborhood of node i , and $\mathbf{a}$ is a weight vector for parameterization.

In the original design of GAT, the authors did not consider the presence of edge features. For this reason, Wang et al. proposed the Edge-Featured Graph Attention Network (EGAT) to solve tasks involving both node and edge features. EGAT comprises two distinct blocks: a node attention block and an edge attention block. These blocks have been designed symmetrically after GAT to enable both node and edge features to update themselves in parallel and in an equivalent manner.

Generative subgraph contrast

GSC, proposed by Han et al. [50], is a self-supervised graph representation learning framework based on contrastive learning. GSC produces contrastive samples by leveraging the inherent topology of graphs while differentiating them through subgraph characteristics and structural patterns. In brief, GSC constructs a subgraph around each center node in the graph and adaptively learns the relationship weights of each node in the subgraph and the center node, thereby enabling the adaptive interpolation of new nodes. The new node is interpolated as follows:

$$\hat{h}_i = \sum_{j=1}^{\mathcal{N}_i} a_j h_j, \quad (3)$$

where h_j represents the embeddings of the neighboring nodes of center node i , and a_j denotes the learned connection weight between the neighboring node j and the center node i . Subsequently, edges are automatically generated between interpolated nodes based on the similarity between nodes. Lastly, the sampled and generated subgraphs are classified as positive sample pairs if they have the same center nodes and negative sample pairs

Table 1 Comparison of representative self-supervised GNN methods relative to SSGMHAN

Method	Main Advantages over SSGMHAN	Main Limitations compared to SSGMHAN
SelfGNN [44]	Excels at modeling short-term collaborative graphs and multi-granular sequential patterns, suitable for strongly temporal user-item interactions.	Designed for recommendation; lacks modeling of edge features, traffic structure, and subgraph geometric differences, making it unsuitable for NID.
TaxoExpan [45]	Incorporates position-enhanced encoding and noise-robust training, effective for hierarchical semantic structures.	Focused on ontology and taxonomy expansion. Does not capture edge features, temporal dynamics, or attack patterns in IoT traffic.
SSA-GAT [46]	Tailored for intrusion detection, combining GAT with contrastive learning to extract node and edge structural patterns.	Does not include multi-head node-edge fusion, WD and GWD-based subgraph geometric contrast, or attention-guided dynamic edge pruning.
SSTD [47]	Constructs positive and negative pairs using local and global traffic differences, suitable for unlabeled IoT NID scenarios.	Relies on handcrafted features and node-centric aggregation; lacks geometric subgraph contrast and fine-grained edge-attention modeling, limiting performance on dynamic networks.

if they have different center nodes. Furthermore, the authors have devised structured contrast loss functions for the proposed model, employing two distinct types of optimal transmission distances.

The proposed method

Overview structure

Figure 1 shows an overview structure of our proposed Self-Supervised Graph Multi-Head Attention Networks. Following data preprocessing, the proposed model comprises two principal components: an encoder module and a self-supervised learning module. In designing the encoder module, it became evident that most existing graph neural network-based intrusion detection methods tend to consider node features and edge features as discrete entities, with a greater focus on node features and the continued use of mean value aggregation for different types of nodes or edges. This obscures the impact of different types of traffic on the learned models. To address this issue, we incorporate a multi-head attention mechanism into the GNN, which enhances the model's ability to recognize intrinsic structural features. In the aggregation phase, weights are adaptively assigned to neighboring nodes and their connecting edges through the multi-head attention mechanism, followed by weighted aggregation to update features. A multilayer perceptron (MLP) then produces the final edge embeddings. In the self-supervised learning module, the Generative Subgraph Comparison (GSC) method of augmented views utilizes the encoder's embedding vectors to capture edge features and local topology, thereby improving the model's intrusion detection performance. In our method, we extend the traditional centralized approach to incorporate a

cloud-edge collaborative network architecture. In this architecture, edge devices handle initial data collection and preprocessing of network traffic, utilizing a subset of features from the NetFlow dataset. These preprocessed data are then transmitted to the cloud for further aggregation, multi-class detection, and global decision-making. The cloud-edge division of labor reduces latency and ensures real-time detection capabilities at the edge, while the cloud performs more complex analysis and decision support. Eventually, we have accomplished the task of multi-classification of malicious traffic using the proposed model. To better adapt to the cloud-edge collaborative system, we also introduced a lightweight consideration. This ensures that the model operates efficiently in real-time, even in resource-constrained edge devices, by optimizing both computational and memory costs.

Data preprocessing

In the data preprocessing stage, the raw NetFlow-formatted data is transformed into graph data for training and testing, as shown in Fig. 2. First, we extract key information from the large volume of NetFlow data in the dataset for each network flow. This information includes the source IP address, destination IP address, source port number, destination port number, IP protocol identifier byte, TCP flags, the number of incoming and outgoing bytes and packets, and the flow duration (in milliseconds). The IP addresses and port numbers are combined into sockets, which serve as the nodes in the graph, while the TCP flags (i.e., the cumulative value of all TCP flags), IP protocol identifier byte, number of bytes, number

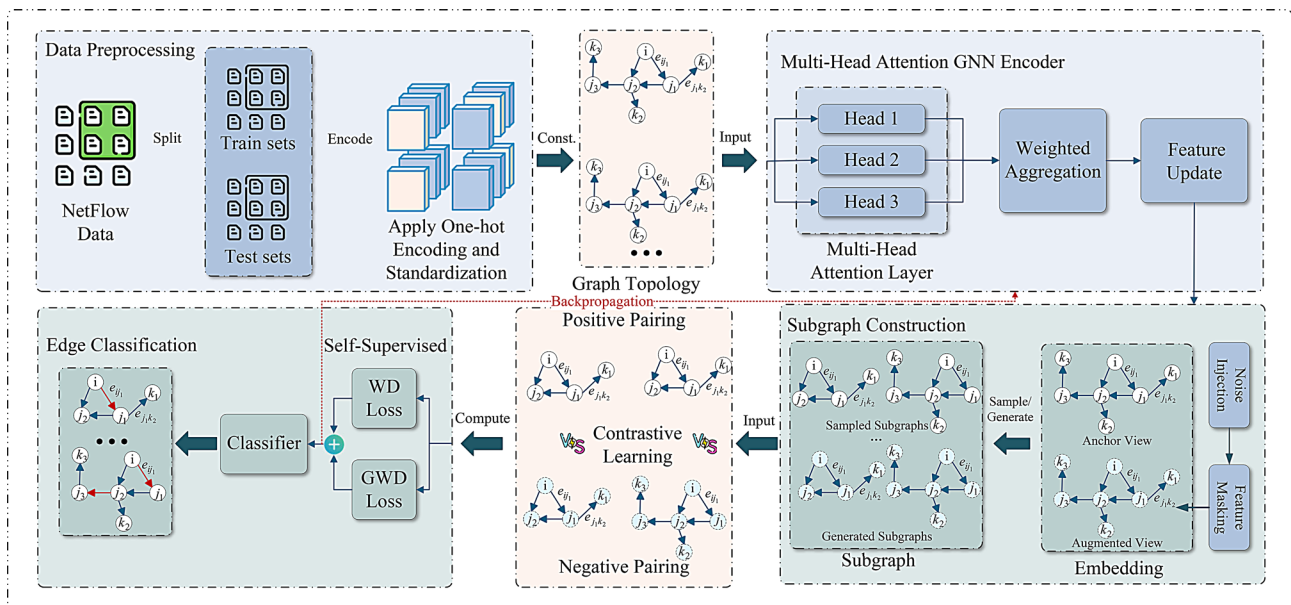


Fig. 1 Overview structure of our proposed SSGMHAN method

	IPv4_SRC_ADOR	IPv4_DST_ADOR	PROTOCOL	L7_PROTO	IN_BYTES	OUT_BYTES	IN_PKTS	OUT_PKTS	TCP_FLAGS	MILLISECONDS	label
0	192.168.100.6:52670	192.168.100.1:53	17	5.212	71	126	1	1	0	4294966	Benign
1	192.168.100.6:49160	192.168.100.149:4444	6	0.000	2177500	199100	4521	4049	24	4176249	Theft
2	192.168.100.46:3456	192.168.100.5:80	6	0.000	8508021	8918372	9086	9086	0	4175916	Benign
3	192.168.100.3:80	192.168.100.55:8080	6	7.000	8442138	9013406	9086	9086	0	4175916	Benign
4	192.168.100.46:80	192.168.100.5:80	6	7.000	8374706	0	9086	0	0	4175916	Benign
...	...	...	...	...	...	...	...	...	...	...	...
600095	192.168.100.46:80	192.168.100.5:80	6	7.000	2330065	0	2523	0	0	4263037	Benign
600096	192.168.100.5:0	192.168.100.3:80	6	0.000	1054423	0	1513	0	0	4263062	Benign
600097	192.168.100.7:365	192.168.100.3:565	17	0.000	62422	0	1357	0	0	4263062	Benign
600098	192.168.100.3:50850	13.54.166.67:8883	6	222.178	11300	1664	32	32	24	4264935	Benign
600099	192.168.100.6:49160	192.168.100.149:4444	6	0.000	40102320	37280	763	590	24	4270068	Theft

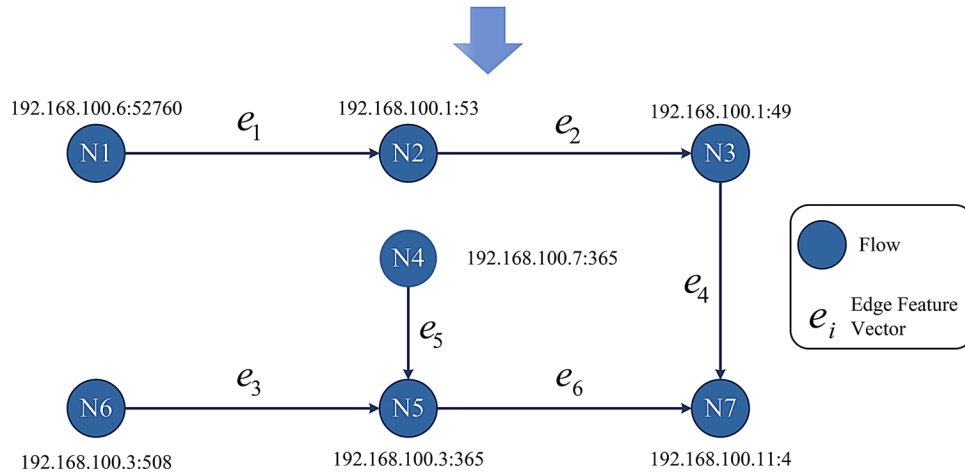


Fig. 2 Converting NetFlow-formatted data into graph representation. Dots represent IP plus port numbers and edges with arrows represent network flows

of packets, and flow duration are considered as edge features.

Second, owing to the considerable size of certain raw datasets, model training on the entire data would consume excessive resources. To address this issue, we applied a class-wise random down-sampling strategy. Specifically, NetFlow records were grouped by the *Attack* label, and 3% of samples were randomly drawn from each class. Categorical features were then encoded as numerical values, and the Standard Scaler function was used to standardize the data, ensuring that feature values were distributed with a mean of 0 and a standard deviation of 1 within a standard normal distribution. This approach prevents the model from being dominated by specific dimensions with disproportionately large feature values. Finally, directionality was assigned to the edges, with network flows directed from source nodes to destination nodes, and the Deep Graph Library (DGL) [51] was utilized to model the graph-structured data.

GNN encoder with multi-head attention

After preprocessing the data, the neural network typically operates as an encoder, transforming the input graph into embeddings that capture essential features relevant to the subsequent task. It is worth noting that in many

graph neural networks, the propagation and aggregation process is primarily focused on nodes, with edges playing a supporting role. Moreover, the majority of aggregation methods utilize mean-value aggregation. However, in the context of NID Systems, where data primarily originates from network flow information between hosts, represented in the constructed graph structure, crucial information is predominantly embedded in edge features rather than node features. Overemphasis on node attributes may result in the neglect of significant edge information. Furthermore, advanced deep learning techniques, such as the attention mechanism, are still infrequently utilized in NIDSs. Despite their effectiveness in capturing node information in established frameworks like Graph Attention Networks (GATs), their application to edges remains under-explored. Therefore, to effectively implement the attention mechanism in GNN-based NID methods, it is crucial to leverage the importance of edge features, thereby providing a more thorough capture of graph structural information, thereby improving the model's overall performance. In this section, we present an edge feature encoder that incorporates a multi-head attention mechanism.

Model initialization

Before aggregating key features sampled from the model, it is necessary to define the inputs and parameters within the model. We define a directed graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the set of nodes and $\mathcal{E}$ represents the set of edges. Each node v_i in $\mathcal{V}$ has a feature vector h_i . There is an edge $e_{ij} = (v_i, v_j)$ in $\mathcal{E}$ with a feature vector e_{ij} between the source node v_j and the destination node v_i . It is important to note that the edge information is obtained by removing the labels, IP addresses, and port properties from the initial network flow data used to establish the graph topology. Therefore, for the node features, we construct a feature tensor of the same shape and initialize it with $[1, \dots, 1]$ (i.e., no initial features are assigned to the nodes). Additionally, the number of graph convolution layers T , the number of attention heads K , and a set of learnable weights matrices W for linearly transforming the input features are defined for adjusting the optimization and implementing the backward propagation.

Sampling and aggregation

In the process of node sampling and aggregation, we utilize a multi-head attention mechanism. In contrast to traditional GAT methods, our model considers the individual source node and its associated edges within the neighborhood when aggregating the features of each target node. Similarly, the computation of the attention weights will also involve these source nodes and edges. The specific process of updating node features is illustrated below:

$$h_i^{t+1} = \left\|_{k=1}^K \left(\sum_{j \in N_i} \left(\alpha_{ij}^{t,k} \cdot [W_k^t \cdot h_j \parallel W_k^t \cdot e_{ij}] \right) \right) \right\|, \quad (4)$$

where α is the attention weight corresponding to different source nodes in the neighborhood of the target node. In order to calculate the attention weights, the multi-head attention mechanism will be employed, considering not only the source and target nodes but also the edges connecting them. For the target node v_i , the weight of each source node v_j and the edge e_{ij} connecting them will be computed, where N_i is the set containing the first-order neighbors of node v_i as well as node v_i itself. In this process, features will be concatenated and parameterized by a vector a . Finally, the attention weights need to be passed through an activation function LeakyReLU and the weights of all possible v_j 's in N_i are normalized using a softmax function. The entire computation procedure can be outlined as follows:

$$\alpha_{ij}^{t,k} = \text{softmax} \left(\text{LeakyReLU} \left((a^k)^T \cdot [W_k^t \cdot h_i \parallel W_k^t \cdot h_j \parallel W_k^t \cdot e_{ij}] \right) \right). \quad (5)$$

In addition, we add a residual connection to the new node features for better error propagation and original feature preservation.

After updating the node features, since a new set of node features has been generated by incorporating edge features into the aggregation, it is necessary to maintain the same shape for both edge and node features at each aggregation step. Accordingly, we also update the edge features, with the resulting new edge features shown below:

$$e_{ij}^{t+1} = \text{CONCAT} (h_j^t, e_{ij}^t). \quad (6)$$

Finally, to obtain the final embedding of the graph $\mathcal{G}$, we will concatenate the node embeddings of the neighboring nodes v_i and v_j to obtain the edge embedding e_{ij} for each edge e_{ij} as follows:

$$e_{ij} = \text{CONCAT} (h_i, h_j), e_{ij} \in \mathcal{E}. \quad (7)$$

Then, the embeddings are fed into the self-supervised module.

Self-supervised learning module

The generative subgraph contrast (GSC) method [50] is a self-supervised approach for learning graph representations by focusing on local structures through contrastive sample generation. GSC begins by randomly selecting central nodes in a graph and then constructs subgraphs around these nodes using breadth-first search [52]. For each sampled subgraph, GSC creates a corresponding generated subgraph. It interpolates new nodes by adaptively learning the relationship weights between nodes within the subgraph and instinctively creates edges between the newly interpolated nodes according to node similarities. Finally, the approach groups both sampled and generated subgraphs into pairs: those sharing the same central node are classified as positive examples, while those with different central nodes are considered negative examples. However, GSC does not adequately address real NID scenarios where edge information is essential, as edges represent actual network flows rather than simple host relationships. In addition, it overlooks the fact that real-world traffic data often contains incomplete or noisy feature measurements, which limits its applicability in practical environments.

Augmented view of random feature-level perturbations

To enhance robustness against the noise and inconsistent measurements prevalent in real network traffic, two complementary views are constructed: an anchor view that preserves the original topology and feature values, and an augmented view that retains the same structure while introducing controlled feature-level disturbances

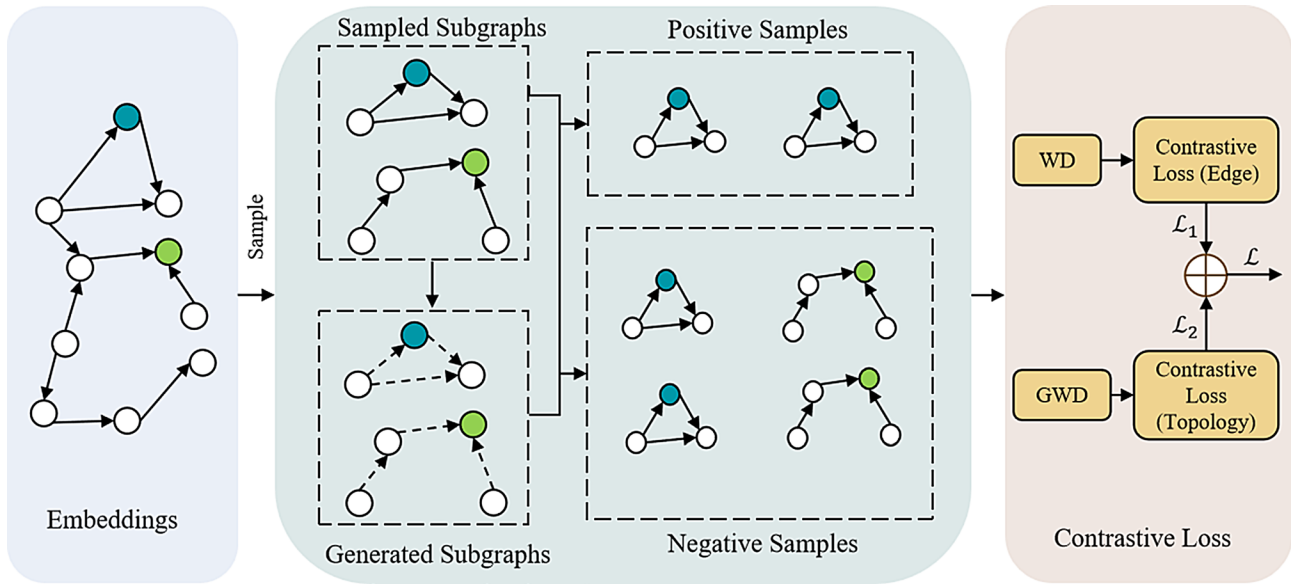


Fig. 3 Structure of the self-supervised learning module

through masking and noise injection. Specifically, the augmented view is generated by applying feature masking and noise injection, allowing the model to learn invariance to incomplete or noisy traffic measurements. In the perturbed view, feature masking is performed by sampling Bernoulli masks to zero out randomly selected feature dimensions, while noise injection is implemented by adding Gaussian perturbations drawn from $(\mathcal{N}(0, \sigma^2))$ to the masked positions. Formally, given an input graph (g) with node features ($\mathbf{X}$) and edge features ($\mathbf{E}$), for a perturbation rate (p):

$$\tilde{\mathbf{X}} = \mathbf{X} \odot (1 - \mathbf{M}_x) + \epsilon_x, \quad \tilde{\mathbf{E}} = \mathbf{E} \odot (1 - \mathbf{M}_e) + \epsilon_e, \quad (8)$$

where $\mathbf{M}_x$ and $\mathbf{M}_e$ are Bernoulli masking matrices for node and edge features, respectively. ϵ_x and ϵ_e are noise matrices drawn from a Gaussian distribution $\mathcal{N}(0, \sigma^2)$, where σ is a predefined noise scale. The operator $\odot$ denotes element-wise multiplication.

Adaptive subgraph generation

To integrate GSC and the GNN encoder with a multi-head attention mechanism for network intrusion detection, we improved GSC by leveraging the characteristics of network traffic and its topology, enabling it to successfully detect differences between various network flows in NIDS, as shown in Fig. 3. In the graph sampling step, central nodes are selected on the anchor view, and for each central node, a subgraph S is constructed comprising the central node and an equal number of its direct neighbors. First, a central node set C is formed by picking a predetermined number of nodes from the graph $\mathcal{G}$. Nodes in C with a degree less than d are then discarded.

Within the sampled subgraph, the distance from any node to the central node is set to 1, and the maximum distance between nodes does not exceed 2. This approach aligns with network attack scenarios, where malicious hosts often target neighboring hosts, while multi-hop attacks represent only a small fraction of network attacks. Particularly, when the number of hops exceeds 2, source address spoofing becomes more prevalent, making it extremely challenging to trace network traffic distant from the host.

Following sampling in the anchor view, view-consistent subgraphs S^g are obtained in the augmented view by projecting the same node sets onto the embedding space of the augmented view. After completing subgraph sampling and projection, a trainable subgraph generation module is employed on augmented view to create positive and negative subgraph samples, with the aim that the generated subgraphs faithfully capture the inherent local structure of the graph. For each node v_i in the subgraph S^g , we interpolate the local structural information of the generated contrastive subgraph G to create a new node $v_{i'}$. The embedding computation procedure for the newly generated nodes is shown below:

$$h_{i'} = \text{ReLU} \left(\sum_{j \in N_i} (\alpha_{ij}) \cdot [h_i || h_j || e_{ij}] \right), \quad (9)$$

where $h_{i'}$ represents the embedding interpolation of nodes in the generated contrastive subgraph G . Incorporating source node embeddings, edge embeddings, and attention weights, we enhance the model's robustness against perturbations and generalization in detecting

new types of abnormal traffic. Furthermore, this method retains the original edges, preserving the original topology while avoiding the additional time complexity that arises from generating edges between similar but unconnected nodes. The framework designates as positive examples the subgraphs sampled from the original view and their counterparts generated in the augmented view using the same node set, and as negative examples the subgraphs centered at different nodes. The former represent two observations of the same semantic instance under relevant feature perturbations, whereas the latter promote discrimination among distinct local patterns.

Contrastive learning

To better measure the geometric differences in edges and graph topology between the sampled and generated subgraphs in positive and negative samples, we use optimal transport distances (specifically, the Wasserstein distance [53] and Gromov-Wasserstein distance [54]) as a similarity metric for the contrastive subgraphs.

The Wasserstein distance (WD) [55] is commonly used to match two discrete distributions, such as two sets of node (or edge) embeddings. For every contrastive pair, edge congruence quantification is performed between originally sampled and artificially generated topological connections. In our setup, WD is used to evaluate the resemblance between sub-edges, calculated as follows:

$$D_w(S, G) = \min_{T \in \pi(\mu, \nu)} \sum_{i=1}^m \sum_{j=1}^m T_{ij} c(e_i, e_j), \quad (10)$$

where $\pi(\mu, \nu)$ represents the joint distribution between all edge pairs from the two subgraphs. $c(e_i, e_j) = \exp(-\varphi(e_i, e_j)/\tau)$ is the cost function evaluating the distance between edges e_i and e_j in subgraphs S and G , respectively. e_i and e_j are edge embeddings.

Based on WD, we define the loss as:

$$\mathcal{L}_1 = \frac{-1}{N(M+1)} \sum_{i=1}^N [\log(\exp(-D_w(S_i, G_i)/\tau)) + \sum_{ni \in \{1, \dots, M\} - \{i\}} \log(1 - \exp(-D_w(S_i, G_{ni})/\tau))] \quad (11)$$

where N is the number of sampled subgraphs, M is the number of negative samples of each subgraph, and τ is a temperature parameter. (S_i, G_i) denotes the positive sample pair, (S_i, G_{ni}) denotes the negative sample pair.

In contrast to WD, the Gromov-Wasserstein distance (GWD) [56] is suitable for the case where only the distance between pairs of nodes in each subgraph is available. GWD allows for the calculation of distances between node pairs within a subgraph and also facilitates

the comparison of these distances between different subgraphs to assess their differences.

Define μ and ν as dual-subgraph discrete distributions, where $\mu = \{\mu_1, \mu_2, \dots, \mu_n\}$ and $\nu = \{\nu_1, \nu_2, \dots, \nu_n\}$, $\sum_{i=1}^n \mu_i = \sum_{j=1}^n \nu_j = 1$, n is the number of subgraph nodes. The GWD similarity between subgraphs can be calculated as:

$$D_{gw}(S, G) = \min_{T \in \pi(\mu, \nu)} \sum_{i, i', j, j'} T_{ij} T_{i'j'} \cdot \hat{c}(h_{S_i}, h_{G_j}, h_{S_{i'}}, h_{G_{j'}}), \quad (12)$$

where the matrix T represents the transport plan between two subgraphs, T_{ij} denotes the amount of mass shifted from μ_i to ν_j . $\hat{c}(h_{S_i}, h_{G_j}, h_{S_{i'}}, h_{G_{j'}}) = \|\hat{c}(h_{S_i}, h_{S_{i'}}) - \hat{c}(h_{G_j}, h_{G_{j'}})\|_2$ serves as the loss function to quantify the edge discrepancy between two subgraphs.

Based on GWD, we define the loss as:

$$\mathcal{L}_2 = \frac{-1}{N(M+1)} \sum_{i=1}^N [\log(\exp(-D_{gw}(S_i, G_i)/\tau)) + \sum_{ni \in \{1, \dots, M\} - \{i\}} \log(1 - \exp(-D_{gw}(S_i, G_{ni})/\tau))] \quad (13)$$

The GWD-based contrastive loss function aims to enhance the edge similarity measurement in positive subgraphs while reducing the resemblance between the connections of negative subgraphs. It effectively captures the geometric differences between subgraphs by focusing on the structural dissimilarities in their edge configurations.

Eventually, we obtain the final loss functions $\mathcal{L}$ as follows:

$$\mathcal{L} = \lambda \mathcal{L}_1 + (1 - \lambda) \mathcal{L}_2, \quad (14)$$

where parameter λ is a hyperparameter used to balance the importance of different loss functions. In this study, λ is set to 0.5, indicating that equal weights are assigned to the multiple loss components being optimized.

Lightweight consideration

To mitigate the intensive computational overhead required by graph convolution operations in resource-constrained cloud-edge environments, we propose a Dynamic Attention Edge Pruning mechanism. Unlike traditional static pruning methods, this approach leverages the learned multi-head attention coefficients as a dynamic indicator of semantic relevance between network flows. The core principle rests on the insight that not all topological connections contribute equally to the detection of malicious patterns; edges with negligible attention weights typically represent redundant

Algorithm 1 Dynamic Attention Edge Pruning Strategy

Input: Current Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; Multi-head attention coefficients $\alpha_{ij}^k, \forall (i, j) \in \mathcal{E}, \forall k \in \{1, \dots, K\}$; Pruning Ratio ρ ; Target Pruning Epoch E_p ; Current Epoch e ;
Output: Optimized Graph structure $\mathcal{G}'$

```

1: if  $e = E_p$  then
2:   Initialize relevance scores set  $\mathcal{S} \leftarrow \emptyset$ 
3:   for each edge  $(i, j) \in \mathcal{E}$  do
4:      $S_{ij} \leftarrow \frac{1}{K} \sum_{k=1}^K \alpha_{ij}^k$  ▷ Aggregate weights across heads
5:     Add  $S_{ij}$  to  $\mathcal{S}$ 
6:   end for
7:   Sort all edges in  $\mathcal{E}$  based on  $\mathcal{S}$  in ascending order
8:   Calculate number of edges to drop:  $N_{drop} \leftarrow \lfloor |\mathcal{E}| \times \rho \rfloor$ 
9:   Identify removal set  $\mathcal{E}_{rem}$  containing top- $N_{drop}$  edges with lowest scores
10:   $\mathcal{E}' \leftarrow \mathcal{E} \setminus \mathcal{E}_{rem}$  ▷ Physical Hard Pruning
11:  Reconstruct adjacency matrix and features for  $\mathcal{E}'$ 
12:   $\mathcal{G}' \leftarrow (\mathcal{V}, \mathcal{E}')$ 
13:  return  $\mathcal{G}'$ 
14: else
15:   return  $\mathcal{G}$  ▷ Keep structure unchanged
16: end if
```

background traffic or noise that dilutes the feature aggregation process. By interpreting these attention distributions as a measure of structural significance, the model allows for the physical removal of edges that fall below a significance threshold during the forward propagation phase. This operation effectively halts the message-passing aggregation for non-informative connections in subsequent layers, transforming the dense interaction graph into a semantically enhanced, sparse structure.

The integration of this mechanism yields a substantial optimization in computational efficiency across diverse network environments. As evidenced by our experiments on the NF-BoT-IoT, NF-BoT-IoT-v2, and NF-ToN-IoT benchmarks, the proposed method consistently achieves a dramatic reduction in training latency by shrinking the adjacency matrix size. In simpler network scenarios, this acceleration is attained with only a marginal, negligible trade-off in detection metrics. Notably, in more complex and heterogeneous environments like NF-ToN-IoT, the pruning process not only drastically cuts processing time but also enhances model generalization by filtering out structural noise. This balance ensures that the intrusion detection system remains lightweight and responsive, satisfying the stringent real-time requirements of edge deployment while maintaining robust threat identification capabilities.

To rigorously formulate the lightweight optimization, the execution logic is detailed in Algorithm 1. This procedure is embedded within the training loop. Unlike soft-masking techniques that merely zero out weights, our approach implements a “hard” structural optimization. When the current epoch e reaches the target pruning epoch E_p , the model aggregates the attention coefficients

α_{ij}^k across all K heads to derive a global relevance score S_{ij} for each network flow. Based on the pruning ratio ρ , the edges with the lowest relevance scores are identified and physically purged from the graph object $\mathcal{E} \setminus \mathcal{E}_{rem}$. The adjacency matrix is subsequently reconstructed to reflect this sparse topology. Crucially, this is a one-time structural intervention; subsequent training epochs operate entirely on the optimized graph $\mathcal{G}'$, thereby permanently reducing the computational load.

Algorithm

The training procedure of the proposed SSGMHAN model (Θ) is listed in Algorithm 2. The input of Algorithm 2 includes the graph structure, node features, edge features, number of attention heads, and number of convolutional layers that need to be defined for the model. Lines 2 to 12 outline the encoding process. Specifically, lines 4 to 7 detail the methods for updating node features and edge features, including the calculation of attention weights and the incorporation of residual connections. Lines 10 to 12 show how the final edge embeddings are obtained through the concatenation of node features. Lines 13 to 16 outline the processes of subgraph sampling and contrastive subgraph generation, constructing positive and negative samples, as well as calculating the loss function and performing backpropagation.

Experiments**Datasets**

Like previous studies [37, 43], we utilized three publicly available NetFlow-formatted benchmark NID datasets in the experiments: NF-BoT-IoT, NF-BoT-IoT-v2, and NF-ToN-IoT. NF-BoT-IoT and NF-BoT-IoT-v2 are datasets

Algorithm 2 SSGMHAN Training

Input: A Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; Input node features $\mathbf{X} = \{\mathbf{h}_i | \forall v_i \in \mathcal{V}\}$; Input edge features $\mathbf{E} = \{\mathbf{e}_{uv} | \forall e_{uv} \in \mathcal{E}\}$; Heads of attention K ; Number of layers T ; Learning rate η ;

Initialize parameters:

- 1: Initialize learnable weights $\Theta = \{W_k^t, \dots\}$ for all $k \in \{1..K\}, t \in \{1..T\}$;

Output: Trained SSGMHAN model parameters Θ

- 2: **for** $epoch = 1$ to $epoch_num$ **do**
- 3: $\mathbf{h}^0 \leftarrow \mathbf{X}; \mathbf{e}^0 \leftarrow \mathbf{E}$
- 4: **for** $t = 1$ to T **do**
- 5: **for** $v_i \in \mathcal{V}$ **do**
- 6: Calculate attention weights $\alpha_{ij}^{t,k}$ using Θ (Eq. (5))
- 7: Aggregate neighborhood to obtain $\mathbf{h}_i^{t+1}$ using Θ (Eq. (4))
- 8: $\mathbf{e}_{ij}^{t+1} \leftarrow \text{CONCAT}(\mathbf{h}_j^t, \mathbf{e}_{ij}^t)$ (Eq. (6))
- 9: **end for**
- 10: **end for**
- 11: **for** $v_i, v_j \in \mathcal{V}$ **do**
- 12: $\mathbf{e}_{ij} \leftarrow \text{CONCAT}(\mathbf{h}_i^T, \mathbf{h}_j^T)$ (Eq. (7))
- 13: **end for**
- 14: Generate an augmented view $\mathcal{G}_{aug}$ and sample a set of anchor subgraphs S from the original graph
- 15: Form positive pairs by generating the counterpart subgraphs G in $\mathcal{G}_{aug}$ sharing the same center nodes
- 16: Form negative pairs by matching each anchor subgraph with randomly sampled subgraphs centered at different nodes
- 17: Calculate loss $\mathcal{L}(\Theta) = 0.5 * \mathcal{L}_1 + 0.5 * \mathcal{L}_2$ (Eq. (13))
- 18: Calculate gradients $\nabla_{\Theta} \mathcal{L}$
- 19: Update model parameters: $\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}$
- 20: **end for**
- 21: **return** Θ

Table 2 Statistics of datasets used in our experiments

Dataset	Class	Sample
NF-BoT-IoT	Benign	13859
	Theft	1909
	DoS	56833
	DDoS	56844
	Reconnaissance	470655
NF-BoT-IoT-v2	Benign	4051
	Theft	73
	DoS	500195
	DDoS	549955
	Reconnaissance	78630
NF-ToN-IoT	Benign	270279
	Backdoor	17247
	DoS	17717
	DDoS	326345
	Injection	468539
	MITM	1295
	Password	156299
	Ransomware	142
	Scanning	21467
	XSS	99944

focused on IoT network traffic, while NF-ToN-IoT is a versatile dataset targeting IoT, covering both normal and attack traffic, and is designed to support research in network anomaly detection and intrusion detection. Table 2 summarizes the key characteristics of the three datasets, including attack categories and feature sets.

- **NF-BoT-IoT:** Australian Centre for Cyber Security developed a realistic network environment containing both normal and botnet traffic [57]. The tools Ostinato and Node-RED were utilized to generate non-IoT and IoT traffic, respectively, creating the BoT-IoT dataset. NF-BoT-IoT [58], derived from BoT-IoT, is a NetFlow-formatted IoT dataset. The dataset consists of 600,100 data flows, including 586,241 attack samples (97.69%, belonging to four types) and 13,859 benign (2.31%) samples.
- **NF-BoT-IoT-v2:** This is a NetFlow-formatted IoT dataset generated by extending the NF-BoT-IoT dataset. Open pcap data underwent feature extraction, and threats were classified. The dataset consists of 37,763,497 data flows, including 37,628,460 (99.64%) attack samples and 135,037 (0.36%) benign samples [59].

Table 3 Performance of SSGMHAN on NF-BoT-IoT and NF-BoT-IoT-v2

Class name	NF-BoT-IoT			NF-BoT-IoT-v2		
	Precision	Recall	F1	Precision	Recall	F1
Benign	100.00	19.20	32.21	100.00	62.50	76.92
Theft	00.00	00.00	00.00	00.00	00.00	00.00
Dos	43.19	47.75	45.35	98.63	95.78	97.18
DDos	38.45	40.14	39.27	99.39	99.39	99.39
Reconnaissance	94.35	95.28	94.81	79.53	95.77	86.90
weighted avg	84.05	83.49	83.12	97.68	97.40	97.46

- **NF-ToN-IoT:** The ToN-IoT dataset [60] is a heterogeneous dataset that includes telemetry data from IoT services, IoT network traffic, and operating system logs. It consists of 22,339,021 flows, including 21,542,641 attack samples and 796,380 benign samples. Sarhan et al. [58] used publicly available pcap files of ToN-IoT to generate its NetFlow format, obtaining NF-ToN-IoT.

Experimental setup

Methods in Comparison: In this study, we evaluate the proposed model in comparison with six existing methods. The models under comparison are NEGSC [43], Anomal-E [40], K-means, and Gaussian Mixture Model (GMM). Among these methods, NEGSC and Anomal-E are more advanced GNN-based self-supervised NID methods that can deal with multi-class prediction. Anomal-E is a method for maximizing the representation of edge information using graph neural networks, to which the authors have added Deep Graph Infomax in order to achieve self-supervised learning. NEGSC is one of the most recent self-supervised learning methods based on graph neural networks, which employs NEGAT as an encoder and NEGSC as a decoder by loss between pairs of sub-subgraphs to train the model. This method has been shown to perform well on multi-classification tasks. Due to the limited research on unsupervised multi-class intrusion detection methods, we have to include two general-purpose classical unsupervised machine learning methods K-means and GMM. Previous studies [61, 62] have shown that K-means and GMM gain the best performance in NID tasks compared with other unsupervised machine learning methods. Lightweight experiments were conducted on a workstation equipped with an Intel(R) Core(TM) i9-14900K processor (32 threads, up to 6.0 GHz) and 32 GB of RAM. Other experiments were conducted on a server consisting of an Intel(R) Xeon(R) Platinum 8336C CPU @ 2.30 GHz.

Evaluation Metrics: Since all the datasets have multiple types of attacks, we perform multi-class classification tasks. For each dataset, we sample it according to different anomaly categories. During the training phase, 70% of the data is used for training. This portion is converted into graph structures, and embeddings are obtained

through an encoder. The encoder is then optimized using a self-supervised learning module. The testing phase employs the residual 30% data portion. For multi-class classification, *Precision*, *Recall*, *Accuracy* and *F1-Score* are the common evaluation metrics for performance evaluation. It is important to note that in a multi-categorization task, *Accuracy* does not fully reflect the model's capabilities due to the large impact of a larger number of samples. And for the remaining three metrics, we obtain its value for each category in a dataset and then use the weighted average of these values as the overall value of the corresponding metric on the dataset.

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \times 100\%, \quad (15)$$

$$Precision = \frac{TP}{TP + FP} \times 100\%, \quad (16)$$

$$Recall = \frac{TP}{TP + FN} \times 100\%, \quad (17)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \times 100\%. \quad (18)$$

Experimental results

Comparison experiments

The first set of experiments evaluates the performance of the proposed SSGMHAN method in each category on every dataset. Table 3 presents the Precision, Recall, and F1-score values that our SSGMHAN achieves on the NF-BoT-IoT and NF-BoT-IoT-v2 datasets. Table 4 shows the values on the NF-ToN-IoT dataset. For dataset NF-BoT-IoT, Precision reached 85.24%, Recall was 83.49%, and F1-score attained 83.46%. In contrast, the NF-BoT-IoT-v2 dataset demonstrated significantly improved results, with Precision at 97.68%, Recall at 97.40%, and F1-score at 97.46%. Notably, the model exhibited high recall, particularly in the DDoS, DoS, and Reconnaissance categories, with substantial improvements across all metrics. However, detection performance for the Theft category remained low on both datasets, likely due to the limited sample size in this class, as indicated by the data imbalance (see Table 2). The performance differences

Table 4 Performance of SSGMHAN on NF-ToN-IoT

Class name	NF-ToN-IoT		
	Precision	Recall	F1
Benign	71.11	76.22	73.58
Backdoor	00.00	00.00	00.00
DoS	00.00	00.00	00.00
DDoS	75.55	55.43	63.94
Injection	47.35	85.89	61.04
MITM	00.00	00.00	00.00
Password	00.00	00.00	00.00
Ransomware	00.00	00.00	00.00
Scanning	00.00	00.00	00.00
XSS	00.00	00.00	00.00
weighted avg	47.89	57.23	50.28

between the datasets suggest that the additional feature attributes in NF-BoT-IoT-v2 enhance the model's ability to better distinguish various traffic categories. In comparison, on the NF-ToN-IoT dataset, precision is 47.89%, recall is 57.23%, and F1-score is 50.28%. Analysis of the performance across different attack categories reveals that most categories show poor detection effectiveness. While the metrics for the Injection and DDoS categories remain relatively stable, the metrics for other categories approach zero. This discrepancy can be attributed to the

class imbalance reflected in the sampling numbers presented in Table 2.

Figure 4 shows the confusion matrix of our model on the NF-BoT-IoT dataset. It can be observed that Benign traffic and Theft traffic were mostly misclassified as Reconnaissance, which may be due to the larger sample size of Reconnaissance traffic in the dataset. This likely led the model to overemphasize its features, resulting in misclassification. Additionally, the Dos and DDos categories were frequently confused, possibly due to the similarity of their features. This confusion matrix suggests that our model may benefit from training on a dataset with more balanced class distributions and larger data volumes.

The second set of experiments compares our SSGMHAN models with some existing unsupervised NID methods. Table 5 lists the outcomes of the comparison experiments conducted on the three datasets. Obviously, our proposed method demonstrates high effectiveness with respect to all three metrics. Specifically, on the NF-BoT-IoT dataset, our method achieves 83.46% F1-score, which outperforms the four unsupervised methods and improves 14.51% over NEGSC, the most advanced method based on self-supervised GNNs. Our proposed method also performs the best on the NF-BoT-IoT-v2

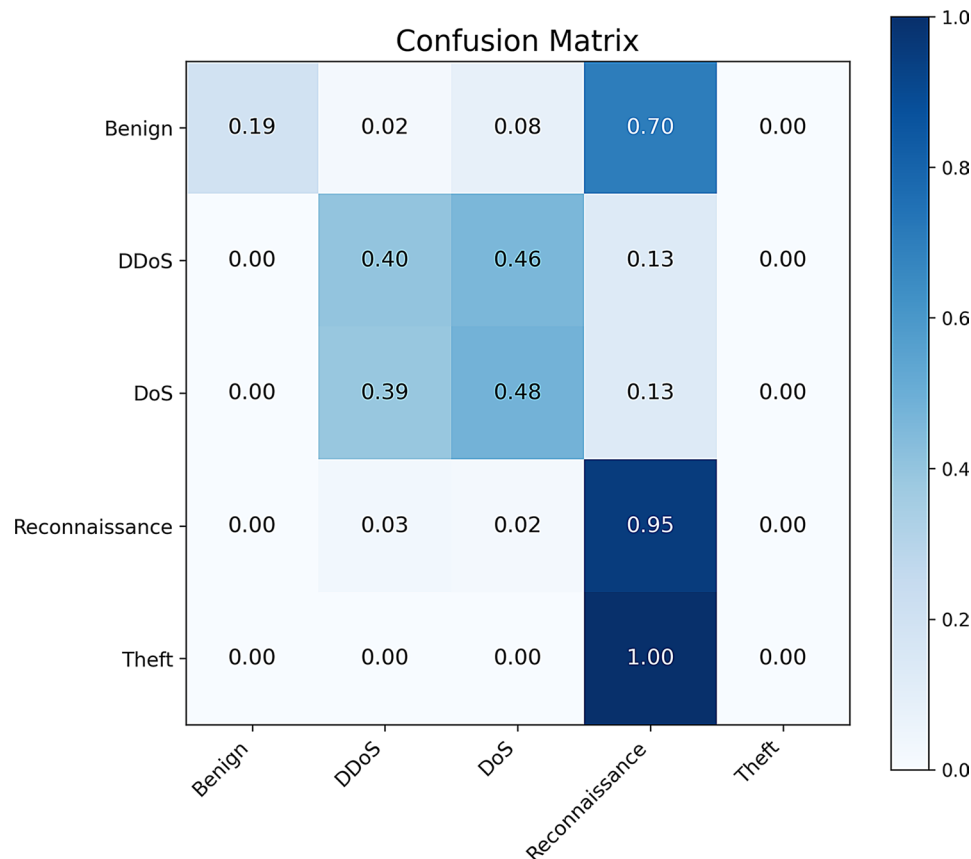
**Fig. 4** Confusion matrix on NF-BoT-IoT

Table 5 Performance comparison among unsupervised methods

Model	Dataset	Precision	Recall	F1	Acc.
SSGMHAN	NF-BoT-IoT	84.05	83.12	83.46	83.49
NEGSC		61.51	78.43	68.95	78.43
Anomal-E		02.20	04.54	02.95	04.54
K-means		02.59	09.29	04.06	09.29
GMM		02.27	06.30	03.10	06.30
SSGMHAN	NF-BoT-IoT-v2	97.68	97.40	97.46	97.40
NEGSC		84.13	90.78	87.32	90.78
Anomal-E		23.99	48.73	32.23	48.73
K-means		59.38	49.71	51.54	49.71
GMM		88.83	48.73	46.16	48.73
SSGMHAN	NF-ToN-IoT	47.89	57.23	50.28	57.23
NEGSC		07.97	20.22	11.39	20.22
Anomal-E		04.02	19.31	06.66	19.31
K-means		11.04	19.66	13.91	19.66
GMM		06.72	02.65	02.78	02.65

Table 6 Ablation experiments

Model	Dataset	Precision	Recall	F1	Acc.
SSGMHAN	NF-BoT-IoT	84.05	83.49	83.12	83.49
SSGMHAN-a		82.88 ^{-1.17}	83.11 ^{-0.38}	82.88 ^{-0.24}	83.11 ^{-0.38}
SSGMHAN-b		83.25 ^{-0.80}	83.06 ^{-0.45}	82.33 ^{-0.79}	83.06 ^{-0.43}
SSGMHAN	NF-BoT-IoT-v2	97.68	97.40	97.46	97.40
SSGMHAN-a		98.14 ^{+0.46}	98.04 ^{+0.64}	98.04 ^{+0.58}	98.04 ^{+0.64}
SSGMHAN-b		95.17 ^{-2.51}	95.20 ^{-2.20}	95.11 ^{-2.35}	95.20 ^{-2.20}
SSGMHAN	NF-ToN-IoT	47.89	57.23	50.28	57.23
SSGMHAN-a		30.81 ^{-17.08}	43.66 ^{-13.57}	31.74 ^{-18.54}	43.66 ^{-13.57}
SSGMHAN-b		41.57 ^{-6.32}	53.70 ^{-3.53}	45.42 ^{-4.86}	53.70 ^{-3.53}

dataset. It achieves 97.46% F1-score, improving 10.14% over NEGSC. On the NF-Ton-IoT dataset, our model consistently outperforms other unsupervised methods. Table 2 shows that NF-Ton-IoT has more categories than the other two datasets. It shows that the greater the number of categories, the more competitive our method becomes. For example, on NF-Ton-IoT our method improves 39.92% over NEGSC in F1-score.

Ablation experiments

This section presents ablation studies designed to evaluate the individual contributions of two key components within the SSGMHAN framework: the generative sub-graph contrastive learning mechanism on augmented views and the multi-head attention mechanism. Table 6 presents the performance of the full SSGMHAN model alongside two variants: SSGMHAN-a, which excludes the augmented-view contrastive learning module, and

SSGMHAN-b, which substitutes multi-head attention with a single-head mechanism.

The results for SSGMHAN-a reveal that the impact of augmented-view contrastive learning is dataset-dependent. On the simpler NF-BoT-IoT and NF-BoT-IoT-v2 datasets, removing this component yielded comparable or even slightly superior performance. Specifically, on NF-BoT-IoT-v2, SSGMHAN-a achieved a peak F1-score of 98.04%, surpassing the full model by 0.64%. This suggests that for relatively straightforward traffic patterns, additional augmentations may introduce unnecessary noise. Conversely, on the complex and heterogeneous NF-ToN-IoT dataset, the absence of this mechanism caused severe performance degradation, with the F1-score dropping by 18.54%. This substantial decline underscores the critical role of augmented-view contrastive learning in enhancing model robustness and generalization capabilities when confronting diverse and intricate network attack scenarios.

In contrast, the results for SSGMHAN-b demonstrate the consistent necessity of the multi-head attention mechanism across all environments. Replacing the multi-head design with a single-head counterpart led to performance degradation across the board. On NF-BoT-IoT, the F1-score decreased by 0.79%. The impact was more pronounced on NF-BoT-IoT-v2 and NF-ToN-IoT, where F1-scores declined by 2.35% and 4.86%, respectively. These consistent reductions confirm that the multi-head attention mechanism is essential for capturing multifaceted relationships within network traffic graphs, enabling the model to simultaneously attend to different subspaces of feature representation, thereby improving overall detection accuracy.

Lightweight experiments

This set of experiments evaluates the effectiveness of the dynamic attention-based edge pruning mechanism by examining its influence on both computational efficiency and detection performance. Tables 7 and 8 provide a comparative analysis of training time and detection metrics before and after integrating this lightweight strategy across the three benchmark datasets. Overall, the results demonstrate that the proposed mechanism consistently yields substantial reductions in training latency while preserving detection capability, and in some scenarios even improving it, across diverse network environments.

As shown in Table 7, the introduction of the pruning mechanism leads to a marked decrease in training overhead for the NF-BoT-IoT and NF-BoT-IoT-v2 datasets. On NF-BoT-IoT, training time is reduced from 68.57 seconds to 33.41 seconds, corresponding to a 51.3% acceleration. This efficiency improvement is accompanied by only a minor performance trade-off, with the weighted average F1-score declining by 1.64%. In contrast, the

Table 7 Training time of SSGMHAN in Seconds on NF-BoT-IoT and NF-BoT-IoT-v2

Class name	NF-BoT-IoT			NF-BoT-IoT-v2		
	Precision	Recall	F1	Precision	Recall	F1
Benign	94.78	12.80	22.56	100.00	37.50	54.55
Theft	20.69	00.75	01.44	00.00	00.00	00.00
Dos	43.77	47.75	45.67	98.43	97.33	97.88
DDos	38.70	70.03	49.85	98.50	99.39	98.94
Reconnaissance	93.53	94.98	94.25	87.84	91.55	89.66
weighted avg(after)	83.15 ^{-0.59}	82.61 ^{-0.49}	80.98 ^{-1.64}	97.73 ^{+0.45}	97.70 ^{+0.25}	97.65 ^{+0.28}
weighted avg(before)	83.74	83.10	82.62	97.28	97.45	97.37
seconds(after)	33.41 ^{-35.16}			8.74 ^{-6.31}		
seconds(before)	68.57			15.05		

Table 8 Training time of SSGMHAN in Seconds on NF-ToN-IoT

Class name	NF-ToN-IoT		
	Precision	Recall	F1
Benign	68.88	73.92	71.31
Backdoor	00.00	00.00	00.00
DoS	00.00	00.00	00.00
DDoS	70.95	72.73	71.83
Injection	55.73	89.34	68.64
MITM	00.00	00.00	00.00
Password	00.00	00.00	00.00
Ransomware	00.00	00.00	00.00
Scanning	00.00	00.00	00.00
XSS	00.00	00.00	00.00
weighted avg(after)	49.22 ^{+2.80}	62.04 ^{+5.60}	54.29 ^{+4.78}
weighted avg(before)	46.42	56.44	49.51
seconds(after)	95.10 ^{-170.12}		
seconds(before)	265.22		

NF-BoT-IoT-v2 dataset exhibits a favorable dual benefit: training time decreases by 41.9%, from 15.05 to 8.74 seconds, while weighted average precision, recall, and F1-score increase by 0.45%, 0.25%, and 0.28%, respectively. These results indicate that, for these datasets, the mechanism not only accelerates computation by simplifying the graph structure but may also remove redundant or noisy information that marginally impedes classification accuracy.

For the heterogeneous NF-ToN-IoT dataset, the findings in Table 8 highlight the most significant advantages of the proposed mechanism. Training time is reduced from 265.22 seconds to 95.10 seconds, representing a 64.1% improvement. Moreover, this reduction in computational cost is accompanied by a notable enhancement in detection performance: the weighted average F1-score increases from 49.51% to 54.29%, while precision and recall improve by 2.80% and 5.60%, respectively. These observations suggest that the NF-ToN-IoT dataset contains substantial structural noise or edge connections that contribute little to the learning process. By removing these low-value interactions, the pruning mechanism functions as a form of regularization that directs the

Table 9 Model parameter size statistics

Module	Encoder layers	Per-head hidden	Attention heads	Node/Edge dims
Encoder	1	8	3	39 / 39
Generator	1	8	3	39 / 39

model toward more informative structural patterns and consequently enhances its generalization performance in complex and resource-limited IoT environments.

Experimental complexity analysis

This section quantifies the per-epoch training complexity of SSMHGNN so that periodic retraining and incremental updates can be scheduled in a cloud-edge collaborative intrusion-detection system. We also report parameter sizes to support capacity and bandwidth planning, as summarized in Table 9. Let N and M be the numbers of nodes and edges in a training graph block. Let F_n and F_e be the node and edge feature dimensions, H the number of attention heads, F_h the per-head hidden size, and L the number of encoder layers. Our implementation uses two isomorphic multi-head edge-aware GAT modules, an Encoder and a Generator. For one forward pass, the dominant time complexity of a single module is $\mathcal{T}_{\text{GAT}} = \mathcal{O}(H \cdot (NF_n F_h + MF_e F_h + MF_h + NF_h))$ which covers node and edge linear projections, per-edge attention scoring with softmax, and message aggregation. The two modules have the same asymptotic cost and are executed in parallel during training, so the per-epoch GNN cost grows linearly with N , M , H , and F_h . Parameter counts increase linearly with F_n and F_e , and we do not use deep stacks, so the total remains lightweight.

Contrastive objectives are computed on subgraphs. Let s be the subgraph size (here $s = 3$), B_s the number of valid subgraphs per epoch ($B_s \leq 20,000$), I_{wd} the number of Sinkhorn iterations for WD, I_g and J the outer and inner iterations for GWD, and d_{edge} the projected edge-feature dimension. The corresponding complexities are $\mathcal{T}_{\text{WD,node}} = \mathcal{O}(3 B_s s^2 (F_h + I_{\text{wd}}))$,

$\mathcal{T}_{WD,edge} = \mathcal{O}(3B_s(s-1)^2(d_{edge} + I_{wd}))$, $\mathcal{T}_{GWD} = \mathcal{O}(3B_s s^2(F_h + I_g J))$. All three scale linearly with B_s . Because s is fixed and small ($s = 3$), this overhead is on the same order as a single GAT forward pass on one graph block.

We qualify the computational efficiency of the model by measuring the average inference latency per batch, as summarized in Table 10. The proposed SSGMHAN significantly outperforms NEGSC in terms of inference speed. For instance, on the NF-BoT-IoT-v2 dataset, our model reduces the latency from 46.44ms to 6.67ms, achieving a speedup factor of over 7. This efficiency is attributed to the architectural design, which decouples complex contrastive loss computations (WD and GWD) from the inference phase. Although the encoder adopts a multi-head attention mechanism, its computational overhead remains modest because the model employs a relatively small embedding dimensionality and a limited number of attention heads. Consequently, SSGMHAN maintains low inference cost and proves highly compatible with the resource constraints of cloud-edge environments, where rapid response capabilities are essential for effective network security.

Incorporating the proposed enhancements introduces marginal computational overhead while significantly reducing the long-term training cost. The Augmented View Contrastive Learning module applies stochastic feature masking and noise injection. These element-wise operations on feature matrices yield a complexity of $\mathcal{T}_{AUG} = \mathcal{O}(NF_n + MF_e)$, which is linear with respect to the graph size and negligible compared to the quadratic matrix multiplications in the GAT layers. The Dynamic Attention Edge Pruning mechanism introduces a topology refinement step. Let E_{prune} denote the epoch where pruning occurs. The overhead at this specific step is dominated by the sorting of attention coefficients to identify non-informative edges, with a complexity of $\mathcal{T}_{prune} = \mathcal{O}(M \log M + M)$, where the linear term accounts for graph reconstruction. However, this is a one-time investment. Post-pruning, the number of edges is reduced to $M' = (1 - \rho)M$, where ρ is the pruning ratio. Consequently, for all subsequent epochs ($E > E_{prune}$), the dominant GNN complexity $\mathcal{T}_{GAT}$ decreases proportionally to the reduction in edge count. Since M is typically much larger than N in network traffic graphs, and ρ is substantial (e.g., $0.3 \sim 0.5$), the cumulative time savings in message aggregation and gradient computation far outweigh the initial sorting cost, ensuring the overall framework remains highly lightweight and scalable.

In summary, both the GNN backbone and the contrastive objectives scale linearly in graph size and attention width, while the model remains lightweight in parameter count. This linear behavior enables predictable retraining

Table 10 Comparison of average inference latency per batch

Method	Dataset	Nodes per Batch	Inference Latency (ms)
SSGMHAN	BoT-IoT	7,647	5.4093 ± 0.5183
	BoT-IoT-v2	2,016	6.6719 ± 1.3750
	ToN-IoT	22,517	5.9720 ± 0.5599
NEGSC	BoT-IoT-v2	2,017	46.4438 ± 4.6230

windows and stable resource budgeting across edge devices and the cloud.

Conclusion

In this paper, we propose a self-supervised Network Intrusion Detection model based on Graph Neural Networks (SSGMHAN), which effectively distinguishes normal network traffic from various types of malicious traffic in an unlabeled data environment by leveraging edge features. The proposed model incorporates a multi-head attention mechanism to account for the weights of distinct edge types and the directional properties within the graph structure. We conducted extensive experiments on multiple NetFlow-based benchmark datasets, comparing our approach with other self-supervised Graph Neural Network methods and traditional unsupervised machine learning clustering techniques. The results highlight the method's superior detection accuracy and practical viability in identifying multi-class network threats.

Specifically, compared to state-of-the-art baselines like NEGSC [43], our analysis confirms that the primary strength of SSGMHAN lies in its multi-head attention mechanism, which captures fine-grained dependencies between specific node types and edge flows that global aggregation methods often overlook. However, despite these strengths, our study identifies certain limitations. First, while the proposed dynamic attention edge pruning strategy ensures lightweight inference suitable for edge devices, the calculation of optimal transport distances during the training phase incurs higher computational overhead than simple reconstruction-based methods. Second, the model exhibits sensitivity to extreme class imbalance. As observed in our experiments, the contrastive learning objective tends to prioritize dominant structural patterns, leading to suboptimal performance for minority attack classes (e.g., Theft or Backdoor).

Consequently, future work should prioritize the innovation of robust intrusion detection architectures capable of addressing these imbalances. We plan to explore techniques such as focal loss or hard sample mining to enhance minority class detection and investigate efficient approximations of optimal transport to further optimize training efficiency.

Abbreviations

DLG	Deep Graph Library
EGAT	Edge-Featured Graph Attention Network
GAT	Graph Attention Network

GMM	Gaussian Mixture Model
GNN	Graph Neural Network
GSC	Generative Subgraph Contrast
GWD	Gromov-Wasserstein Distance
ML	Machine Learning
MLP	Multi-Layer Perceptron
NEGSC	NetFlow-Edge Generative Subgraph Contrast
NID	Network Intrusion Detection
NIDS	Network Intrusion Detection System
SSGMHAN	Self-Supervised Graph Multi-Head Attention Network
SSL	Self-Supervised Learning
WD	Wasserstein Distance

Author contributions

Haiyang Diao: Conceptualization, Data curation, Methodology, Writing - original draft. Xiang Li: Investigation, Methodology, Validation, Writing - original draft. Wan Zhang: Investigation, Methodology, Writing - original draft. Yan Sibyl Yu: Validation. Ye Wang: Validation, Writing - review & editing. Jing Zhang: Funding acquisition, Supervision, Writing - review & editing.

Funding

This work was funded by Basic Research Program of Jiangsu [Grant No. BK20251747], the National Natural Science Foundation of China [Grant No. 62076130], the Southeast University Start-up Research Fund [Grant No. RF1028623059], and the Big Data Computing Center of Southeast University.

Data availability

The datasets used and/or analysed during the current study are available from the corresponding author on reasonable request.

Declarations

Consent for publication

All authors agree to publish the research in this journal.

Competing interests

The authors declare no competing interests.

Received: 6 November 2025 / Accepted: 15 January 2026

Published online: 31 January 2026

References

- Wu T, Dou W, Wu F, Tang S, Hu C, Chen J (2016) A deployment optimization scheme over multimedia big data for large-scale media streaming application. *ACM Trans Multimedia Comput Commun Appl (TOMM)* 12(5s):1–23
- Gu R, Wang S, Dai H, Chen X, Wang Z, Bao W, Zheng J, Tu Y, Huang Y, Qi L et al (2024) Fluid-shuttle: efficient cloud data transmission based on serverless computing compression. In *IEEE/ACM Transactions on Networking*
- Dou W, Tang W, Wu X, Qi L, Xu X, Zhang X, Hu C (2020) An insurance theory based optimal cyber-insurance contract against moral hazard. *Inf Sci* 527:576–589
- Fei F, Li S, Dai H, Hu C, Dou W, Ni Q (2017) A k-anonymity based schema for location privacy preservation. *IEEE Trans Sustain Comput* 4(2):156–167
- Qi L, Wang R, Hu C, Li S, He Q, Xu X (2019) Time-aware distributed service recommendation with privacy-preservation. *Inf Sci* 480:354–364
- Qi L, Xu X, Wu X, Ni Q, Yuan Y, Zhang X (2023) Digital-twin-enabled 6g mobile network video streaming using mobile crowdsourcing. *IEEE J Sel Areas Commun* 41(10):3161–3174
- Liu Y, Zhou X, Kou H, Zhao Y, Xu X, Zhang X, Qi L (2024) Privacy-preserving point-of-interest recommendation based on simplified graph convolutional network for geological traveling. *ACM Trans Intell Syst Technol* 15(4):1–17
- Dutta S, Mawla T, Rabbi MF (2019) A comparison study of temporal signature mining over traditional data mining techniques to detect network intrusion, in *Proceedings of 2018 emerging technologies in data mining and information security* pp 757–763
- Díaz-Verdejo J, Muñoz-Calle J, Alonso AE, Alonso RE, Madinabeitia G (2022) On the detection capabilities of signature-based intrusion detection systems in the context of web attacks. *Appl Sci* 12(2):852
- Aldweesh A, Derhab A, Emam AZ (2020) Deep learning approaches for anomaly-based intrusion detection systems: a survey, taxonomy, and open issues. *Knowl-Based Syst* 189:105124
- Abbasi M, Shahraki A, Taherkordi A (2021) Deep learning for network traffic monitoring and analysis (ntma): a survey. *Comput Commun* 170:19–41
- Guerra JL, Catania C, Veas E (2022) Datasets are not enough: challenges in labeling network traffic. *Comput Secur* 120:102810
- Attou H, Guezzaz A, Benkirane S, Azrou M, Farhaoui Y (2023) Cloud-based intrusion detection approach using machine learning techniques. *Big Data Min Analytics* 6(3):311–320
- FWei HL, Zhao Z, Hu H (2023) xNIDS: explaining Deep learning-based network intrusion detection systems for active intrusion responses, in *32nd USENIX security symposium* 4337–4354
- Rieger P, Chilese M, Mohamed R, Miettinen M, Fereidooni H, Sadeghi AR (2023) ARGUS: Context-based detection of stealthy IoT infiltration attacks, in *32nd USENIX security symposium* 4301–4318
- Gori M, Monfardini G, Scarselli F (2005) A new model for learning in graph domains. In *2005 IEEE International Joint Conference on Neural Networks*, vol 2, pp 729–734
- Pujol-Perich D, Suárez-Varela J, Cabellos-Aparicio A, Barlet-Ros P (2022) Unveiling the potential of graph neural networks for robust intrusion detection. *ACM SIGMETRICS Perform Evaluation Rev* 49(4):111–117
- Guo W, Qiu H, Liu Z, Zhu J, Wang Q (2022) Gld-net: deep learning to detect ddos attack via topological and traffic feature fusion. *Comput Intel Neurosci* 2022(1):4611331
- Qi L, Lin W, Zhang X, Dou W, Xu X, Chen J (2022) A correlation graph based approach for personalized and compatible web apis recommendation in mobile app development. *IEEE Trans Knowl Data Eng* 35(6):5444–5457
- Wu S, Shen S, Xu X, Chen Y, Zhou X, Liu D, Xue X, Qi L (2022) Popularity-aware and diverse web apis recommendation based on correlation graph. *IEEE Trans Comput Soc Syst* 10(2):771–782
- Iliofotou M, Pappu P, Faloutsos M, Mitzenmacher M, Singh S, Varghese G (2007) Network monitoring using traffic dispersion graphs (tdgs). In *Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement*, pp 315–320
- Chen L, Li Y, Liu H, Guo H (2024) Node and edge joint embedding for heterogeneous information network. *Big Data Min Analytics* 7(3):730–752
- Gilmer J, Schoenholz SS, Riley PF, Vinyals O, Dahl GE (2017) Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pp 1263–1272
- Hamilton W, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. *Adv Neural Inf Process Syst* 30:1025–1035
- Zhang H, Qi G, Sun P, Dong S, Liu X, Tan J (2025) Decision-making approach for complex network defense based on flipit game. *Tsinghua Sci Technol*. <https://www.sciopen.com/article/10.26599/TST.2024.9010206>
- Xiao Y, Ma Z, Huang W, Qiao C, Zhao B, Zhang D, Pei Q (2025) Pure-gnn: a lightweight purified graph neural network against adversarial attacks. *Tsinghua Science and Technology*
- Lei K, Fu Q, Ni J, Wang F, Yang M, Xu K (2019) Detecting malicious domains with behavioral modeling and graph embedding. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, (IEEE), pp 601–611
- Diao H, Li X, Jiang X, Zhang J (2025) Self-supervised graph multi-head attention networks for network intrusion detection. In *Proceedings of the 22nd IEEE International Conference on Ubiquitous Intelligence and Computing (UIC)*
- Mohy-Eddine M, Guezzaz A, Benkirane S, Azrou M, Farhaoui Y (2023) An ensemble learning based intrusion detection model for industrial iot security. *Big Data Min Analytics* 6(3):273–287
- Eljaily A, Uddin MY, Ahmad S (2024) Novel framework for an intrusion detection system using multiple feature selection methods based on deep learning. *Tsinghua Sci Technol* 29(4):948–958
- Song HM, Kim HK (2021) Self-supervised anomaly detection for in-vehicle network using noised pseudo normal data. *IEEE Trans Veh Technol* 70(2):1098–1108
- Wang W, Jian S, Tan Y, Wu Q, Huang C (2023) Robust unsupervised network intrusion detection with self-supervised masked context reconstruction. *Comput Secur* 128:103131
- Geng H, Zhang Z, Qian Y, Yang B, Ma Q, Chi H, Yang J, Yin X (2025) Caeif: an anomaly detection approach based on temporal representation of the reconstruction error. *Tsinghua Science and Technology*

34. Zhou J, Cui G, Hu S, Zhang Z, Yang C, Liu Z, Wang L, Li C, Sun M (2020) Graph neural networks: a review of methods and applications. *AI Open* 1:57–81
35. Zhou J, Xu Z, Rush AM, Yu M (2020) Automating botnet detection with graph neural networks. *arXiv preprint arXiv:2003.06344*. <https://arxiv.org/abs/2003.06344>
36. Xiao Q, Liu J, Wang Q, Jiang Z, Wang X, Yao Y (2020) Towards network anomaly detection using graph embeddings. In *Proceedings of 20th International Conference on Computational Science*, pp 156–169
37. Lo WW, Layeghy S, Sarhan M, Gallagher M, Portmann M (2022) E-Graphsage: a graph neural network based intrusion detection system for IoT. In *NOMS 2022–2022 IEEE/IFIP Network Operations and Management Symposium*, pp 1–9
38. Zhou X, Liang W, Li W, Yan K, Shimizu S, Kevin I, Wang K (2021) Hierarchical adversarial attacks against graph-neural-network-based IoT network intrusion detection system. *IEEE Internet Things J* 9(12):9310–9319
39. Gong Z, Chen S, Dai Q, Feng Y, Wang J, Zhang J (2025) Scoamps: semi-supervised graph contrastive learning based on associative memory network and pseudo-label similarity. *Big Data Min Analytics* 8(2):273–291
40. Caville E, Lo WW, Layeghy S, Portmann M (2022) Anomal-e: a self-supervised network intrusion detection system based on graph neural networks. *Knowl-Based Syst* 258:110030
41. Nguyen H, Kashaf R (2023) Ts-ids: traffic-aware self-supervised learning for IoT network intrusion detection. *Knowl-Based Syst* 279:110966
42. Xie L, Ye M, Chen B (2023) A network intrusion detection system based on self-supervised co-contrastive learning. In *International Conference on Network Simulation and Evaluation*, pp 387–399
43. Xu R, G.Wu WW, Gao X, He A, Zhang Z (2024) Applying self-supervised learning to network intrusion detection for network flows with graph neural network. *Comput Networks* 248:110495
44. Liu Y, Xia L, Huang C (2024) Selfgnn: self-supervised graph neural networks for sequential recommendation, in *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval* pp 1609–1618
45. Shen J, Shen Z, Xiong C, Wang C, Wang K, Han J (2020) TaxoExpan: self-supervised taxonomy expansion with position-enhanced graph neural network. In *Proceedings of the web conference 2020*, pp 486–497
46. Liu Q, Zhang H, Zhang Y, Fan L, Jin X (2024) SSA-GAT: graph-based self-supervised learning for network intrusion detection. In *International Conference on Artificial Neural Networks*, Springer, pp 476–491
47. Chen Z, Zou H, Hu T, Fang X, Zheng J, Li J, Pan Y (2025) A network intrusion detection system based on self-supervised learning of traffic differentiation in internet of things. *Eng Appl Artif Intel* 160:111973
48. Velickovic P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y et al (2017) Graph attention networks. *STAT* 1050(20):10–48550
49. Wu Z, Pan S, Chen F, Long G, Zhang C, Philip SY (2020) A comprehensive survey on graph neural networks. *IEEE Trans Neural Networks Learn Syst* 32(1):4–24
50. Han Y, Hui L, Jiang H, Qian J, Xie J (2022) Generative subgraph contrast for self-supervised graph representation learning. In *European Conference on Computer Vision*, pp 91–107
51. Wang MY (2019) Deep graph library: towards efficient and scalable deep learning on graphs. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*
52. Bundy A, Wallen L (1984) Breadth-first search. *Catalogue Artif Intel Tools* 13–13
53. Panaretos VM, Zemel Y (2019) Statistical aspects of wasserstein distances. *Annu Rev Stat Its Application* 6(1):405–431
54. Mémoli F (2011) Gromov–wasserstein distances and the metric approach to object matching. *Found Comput Math* 11:417–487
55. Chen L, Gan Z, Cheng Y, Li L, Carin L, Liu J (2020) Graph optimal transport for cross-domain alignment. In *International Conference on Machine Learning*, pp 1542–1553
56. Peyré G, Cuturi M, Solomon J (2016) Gromov–wasserstein averaging of kernel and distance matrices. In *International Conference on Machine Learning*, PMLR, pp 2664–2672
57. Livadas C, Walsh R, Lapsley D, Strayer WT (2006) Using machine learning techniques to identify botnet traffic. In *31st IEEE Conference on Local Computer Networks*, pp 967–974
58. Sarhan M, Layeghy S, Moustafa N, Portmann M (2021) Netflow datasets for machine learning-based network intrusion detection systems. In *Big Data Technologies and Applications: 10th EAI International Conference, BDTA 2020, and 13th EAI International Conference on Wireless Internet, WiCON 2020*, pp 117–135
59. Sarhan M, Layeghy S, Portmann M (2022) Towards a standard feature set for network intrusion detection system datasets. *Mob Networks Appl* 1–14
60. Moustafa N (2019) Ton-iot datasets. <https://doi.org/10.21227/fesz-dm97>
61. Bitaab M, Hashemi S (2017) Hybrid intrusion detection: combining decision tree and gaussian mixture model. In *2017 14th International ISC (Iranian Society of Cryptology) Conference on Information Security and Cryptology (ISCISC)*, IEEE, pp 8–12
62. Aung YY, Min MM (2018) An analysis of k-means algorithm based network intrusion detection system. *Adv Sci, Technol Eng Syst J* 3(1):496–501

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.